

SOULYATRI VOICE AI - Final Build Bible

Clean-room plan to build an emotional, low-latency, code-switched voice AI

Prepared for Dhruv Paleja / Soulyatri

2026-05-14

Contents

1 SOULYATRI VOICE AI - Final Build Bible	7
1.1 One-page decision	8
2 0. Evidence and assumptions	8
2.1 0.1 What I trust	8
2.2 0.2 What I do not trust	8
2.3 0.3 User-provided Vatsal Bharti resume context	8
2.4 0.4 Uploaded documents integrated	9
3 1. Research conclusions	9
3.1 1.1 Rumik Silk category analysis	9
3.2 1.2 Sesame CSM conclusion	9
3.3 1.3 Moshi/Mimi conclusion	9
3.4 1.4 Hume conclusion	9
3.5 1.5 OpenVoice / voice cloning conclusion	9
3.6 1.6 The winning thesis	9
4 2. Product spec	10
4.1 2.1 Product modes	10
4.1.1 Companion mode	10
4.1.2 Creator mode	10
4.1.3 Agent mode	10
4.1.4 Studio mode	10
4.1.5 Edge mode	10
5 3. Architecture	10
5.1 3.1 Two-track architecture	10
5.2 3.2 Runtime flow	11
5.3 3.3 Model stack	11
6 4. Metrics and evaluation	11
6.1 4.1 Evaluation harness requirements	12
6.2 4.2 Eval JSON schema	12
7 5. Repo skeleton	12
8 6. Agent execution protocol	13
8.1 6.1 Rules for Claude Code / Codex / Cursor	13
8.2 6.2 Universal agent prompt	14
8.3 6.3 Division of labor	14

9	7. Module build cards	14
9.1	7.1 Module: codec - Mimi adapter and future Mimi-Lite	14
9.2	7.2 Module: tokenizer - PCS-Tok++ phonetic code-switch tokenizer	15
9.3	7.3 Module: data - Synthetic + real data flywheel	15
9.4	7.4 Module: model - CSM/Moshi adapters, emotion FiLM, speaker conditioning	16
9.5	7.5 Module: training - SFT, DPO, GRPO and smoke tests	16
9.6	7.6 Module: inference - Streaming server and latency engine	17
9.7	7.7 Module: eval - Voice quality, code-switch, emotion, safety metrics	18
9.8	7.8 Module: safety - Consent, watermarking, policy gates	18
10	8. Data engine	19
10.1	8.1 Dataset tiers	19
10.2	8.2 Synthetic Hinglish prompt template	19
10.3	8.3 Real recording SOP	20
10.4	8.4 Data manifest schema	20
11	9. Training plan	20
11.1	9.1 Day-60 training sequence	20
11.2	9.2 Hyperparameter starting defaults	20
11.3	9.3 DPO pair schema	21
12	10. Latency engineering	21
12.1	10.1 Budget	21
12.2	10.2 Optimization order	21
12.3	10.3 Latency tracer event list	21
13	11. Safety and consent	22
13.1	11.1 Non-negotiable rules	22
13.2	11.2 Consent gate pseudocode	22
14	12. Phase-by-phase execution	22
14.1	Phase 0: Reality-lock and target definition	22
14.2	Phase 1: Repo, environment, and baseline demo	23
14.3	Phase 2: Evaluation harness	24
14.4	Phase 3: Emotion grammar and text data engine	24
14.5	Phase 4: Consent-first real recording micro-set	25
14.6	Phase 5: Tag-only LoRA SFT	26
14.7	Phase 6: Hinglish/code-switch SFT v2	27
14.8	Phase 7: Streaming product shell	27
14.9	Phase 8: DPO-lite alignment	28
14.10	Phase 9: Persona and voice conditioning	29
14.11	Phase 10: Safety, watermark, and abuse defense	30
14.12	Phase 11: 60-day launch demo	30
14.13	Phase 12: 16-week v1 scale-up	31
14.14	Phase 13: 9-month frontier path	32
15	13. Day-by-day 60-day calendar	33
15.1	Day 1 - Week 1	33
15.2	Day 2 - Week 1	33
15.3	Day 3 - Week 1	33
15.4	Day 4 - Week 1	33
15.5	Day 5 - Week 1	34
15.6	Day 6 - Week 1	34
15.7	Day 7 - Week 1	34
15.8	Day 8 - Week 2	34
15.9	Day 9 - Week 2	34
15.10	Day 10 - Week 2	34
15.11	Day 11 - Week 2	35

15.12	Day 12 - Week 2	35
15.13	Day 13 - Week 2	35
15.14	Day 14 - Week 2	35
15.15	Day 15 - Week 3	35
15.16	Day 16 - Week 3	36
15.17	Day 17 - Week 3	36
15.18	Day 18 - Week 3	36
15.19	Day 19 - Week 3	36
15.20	Day 20 - Week 3	36
15.21	Day 21 - Week 3	36
15.22	Day 22 - Week 4	37
15.23	Day 23 - Week 4	37
15.24	Day 24 - Week 4	37
15.25	Day 25 - Week 4	37
15.26	Day 26 - Week 4	37
15.27	Day 27 - Week 4	38
15.28	Day 28 - Week 4	38
15.29	Day 29 - Week 5	38
15.30	Day 30 - Week 5	38
15.31	Day 31 - Week 5	38
15.32	Day 32 - Week 5	38
15.33	Day 33 - Week 5	39
15.34	Day 34 - Week 5	39
15.35	Day 35 - Week 5	39
15.36	Day 36 - Week 6	39
15.37	Day 37 - Week 6	39
15.38	Day 38 - Week 6	40
15.39	Day 39 - Week 6	40
15.40	Day 40 - Week 6	40
15.41	Day 41 - Week 6	40
15.42	Day 42 - Week 6	40
15.43	Day 43 - Week 7	40
15.44	Day 44 - Week 7	41
15.45	Day 45 - Week 7	41
15.46	Day 46 - Week 7	41
15.47	Day 47 - Week 7	41
15.48	Day 48 - Week 7	41
15.49	Day 49 - Week 7	42
15.50	Day 50 - Week 8	42
15.51	Day 51 - Week 8	42
15.52	Day 52 - Week 8	42
15.53	Day 53 - Week 8	42
15.54	Day 54 - Week 8	42
15.55	Day 55 - Week 8	43
15.56	Day 56 - Week 8	43
15.57	Day 57 - Week 9	43
15.58	Day 58 - Week 9	43
15.59	Day 59 - Week 9	43
15.60	Day 60 - Week 9	44

16 14. File-by-file implementation backlog	44
16.1 Backlog item 1: soulyatri/codec/mimi_adapter.py	44
16.2 Backlog item 2: soulyatri/codec/rvq_dropout.py	44
16.3 Backlog item 3: soulyatri/codec/codecs_eval.py	44
16.4 Backlog item 4: soulyatri/tokenizer/pcs_tok.py	45
16.5 Backlog item 5: soulyatri/tokenizer/phoneme_anchor.py	45
16.6 Backlog item 6: soulyatri/tokenizer/emotion_grammar.py	45

16.7	Backlog item 7: soulyatri/data/schema.py	46
16.8	Backlog item 8: soulyatri/data/synth_hinglish.py	46
16.9	Backlog item 9: soulyatri/data/filter_audio.py	46
16.10	Backlog item 10: soulyatri/data/build_lhotse.py	47
16.11	Backlog item 11: soulyatri/data/build_webdataset.py	47
16.12	Backlog item 12: soulyatri/model/emotion_film.py	47
16.13	Backlog item 13: soulyatri/model/speaker_cond.py	48
16.14	Backlog item 14: soulyatri/model/csm_lora.py	48
16.15	Backlog item 15: soulyatri/model/duplex_streams.py	48
16.16	Backlog item 16: soulyatri/training/stage00_baseline.py	49
16.17	Backlog item 17: soulyatri/training/stage01_lora_tags.py	49
16.18	Backlog item 18: soulyatri/training/stage02_hinglish_sft.py	49
16.19	Backlog item 19: soulyatri/training/stage03_emotion_sft.py	50
16.20	Backlog item 20: soulyatri/training/stage04_dpo.py	50
16.21	Backlog item 21: soulyatri/training/stage05_grpo.py	50
16.22	Backlog item 22: soulyatri/inference/server.py	51
16.23	Backlog item 23: soulyatri/inference/livekit_agent.py	51
16.24	Backlog item 24: soulyatri/inference/pipecat_pipeline.py	51
16.25	Backlog item 25: soulyatri/inference/interrupt_monitor.py	51
16.26	Backlog item 26: soulyatri/inference/latency_tracer.py	52
16.27	Backlog item 27: soulyatri/eval/wer.py	52
16.28	Backlog item 28: soulyatri/eval/mos.py	52
16.29	Backlog item 29: soulyatri/eval/emotion_accuracy.py	53
16.30	Backlog item 30: soulyatri/eval/code_switch.py	53
16.31	Backlog item 31: soulyatri/eval/speaker_similarity.py	53
16.32	Backlog item 32: soulyatri/eval/human_ab.py	54
16.33	Backlog item 33: soulyatri/safety/consent.py	54
16.34	Backlog item 34: soulyatri/safety/watermark.py	54
16.35	Backlog item 35: soulyatri/safety/policy.py	55
17	15. Copy-paste commands	55
17.1	15.1 Bootstrap	55
17.2	15.2 Baseline eval	55
17.3	15.3 LoRA SFT	55
17.4	15.4 Serve local demo	56
18	16. Budget	56
19	17. Risk register	56
20	18. Source index	57
21	19. Final operating principle	57
22	Appendix A. Ultra-granular phase subtasks	57
22.1	A.0 Reality-lock and target definition	57
22.1.1	A.0.1 Subtask block	57
22.1.2	A.0.2 Subtask block	58
22.1.3	A.0.3 Subtask block	58
22.1.4	A.0.4 Subtask block	58
22.1.5	A.0.5 Subtask block	59
22.1.6	A.0.6 Subtask block	59
22.1.7	A.0.7 Subtask block	59
22.1.8	A.0.8 Subtask block	59
22.1.9	A.0.9 Subtask block	60
22.1.10	A.0.10 Subtask block	60
22.2	A.1 Repo, environment, and baseline demo	60
22.2.1	A.1.1 Subtask block	60

22.2.2 A.1.2 Subtask block	61
22.2.3 A.1.3 Subtask block	61
22.2.4 A.1.4 Subtask block	61
22.2.5 A.1.5 Subtask block	62
22.2.6 A.1.6 Subtask block	62
22.2.7 A.1.7 Subtask block	62
22.2.8 A.1.8 Subtask block	62
22.2.9 A.1.9 Subtask block	63
22.2.10A.1.10 Subtask block	63
22.3 A.2 Evaluation harness	63
22.3.1 A.2.1 Subtask block	63
22.3.2 A.2.2 Subtask block	64
22.3.3 A.2.3 Subtask block	64
22.3.4 A.2.4 Subtask block	64
22.3.5 A.2.5 Subtask block	64
22.3.6 A.2.6 Subtask block	64
22.3.7 A.2.7 Subtask block	64
22.3.8 A.2.8 Subtask block	65
22.3.9 A.2.9 Subtask block	65
22.3.10A.2.10 Subtask block	65
22.4 A.3 Emotion grammar and text data engine	65
22.4.1 A.3.1 Subtask block	65
22.4.2 A.3.2 Subtask block	65
22.4.3 A.3.3 Subtask block	66
22.4.4 A.3.4 Subtask block	66
22.4.5 A.3.5 Subtask block	66
22.4.6 A.3.6 Subtask block	67
22.4.7 A.3.7 Subtask block	67
22.4.8 A.3.8 Subtask block	67
22.4.9 A.3.9 Subtask block	68
22.4.10A.3.10 Subtask block	68
22.5 A.4 Consent-first real recording micro-set	68
22.5.1 A.4.1 Subtask block	68
22.5.2 A.4.2 Subtask block	68
22.5.3 A.4.3 Subtask block	69
22.5.4 A.4.4 Subtask block	69
22.5.5 A.4.5 Subtask block	69
22.5.6 A.4.6 Subtask block	70
22.5.7 A.4.7 Subtask block	70
22.5.8 A.4.8 Subtask block	70
22.5.9 A.4.9 Subtask block	71
22.5.10A.4.10 Subtask block	71
22.6 A.5 Tag-only LoRA SFT	71
22.6.1 A.5.1 Subtask block	71
22.6.2 A.5.2 Subtask block	71
22.6.3 A.5.3 Subtask block	72
22.6.4 A.5.4 Subtask block	72
22.6.5 A.5.5 Subtask block	72
22.6.6 A.5.6 Subtask block	72
22.6.7 A.5.7 Subtask block	72
22.6.8 A.5.8 Subtask block	72
22.6.9 A.5.9 Subtask block	73
22.6.10A.5.10 Subtask block	73
22.7 A.6 Hinglish/code-switch SFT v2	73
22.7.1 A.6.1 Subtask block	73
22.7.2 A.6.2 Subtask block	73
22.7.3 A.6.3 Subtask block	74

22.7.4	A.6.4 Subtask block	74
22.7.5	A.6.5 Subtask block	74
22.7.6	A.6.6 Subtask block	75
22.7.7	A.6.7 Subtask block	75
22.7.8	A.6.8 Subtask block	75
22.7.9	A.6.9 Subtask block	75
22.7.10	A.6.10 Subtask block	76
22.8	A.7 Streaming product shell	76
22.8.1	A.7.1 Subtask block	76
22.8.2	A.7.2 Subtask block	76
22.8.3	A.7.3 Subtask block	77
22.8.4	A.7.4 Subtask block	77
22.8.5	A.7.5 Subtask block	77
22.8.6	A.7.6 Subtask block	78
22.8.7	A.7.7 Subtask block	78
22.8.8	A.7.8 Subtask block	78
22.8.9	A.7.9 Subtask block	78
22.8.10	A.7.10 Subtask block	79
22.9	A.8 DPO-lite alignment	79
22.9.1	A.8.1 Subtask block	79
22.9.2	A.8.2 Subtask block	79
22.9.3	A.8.3 Subtask block	79
22.9.4	A.8.4 Subtask block	80
22.9.5	A.8.5 Subtask block	80
22.9.6	A.8.6 Subtask block	80
22.9.7	A.8.7 Subtask block	80
22.9.8	A.8.8 Subtask block	80
22.9.9	A.8.9 Subtask block	80
22.9.10	A.8.10 Subtask block	81
22.10	A.9 Persona and voice conditioning	81
22.10.1	A.9.1 Subtask block	81
22.10.2	A.9.2 Subtask block	81
22.10.3	A.9.3 Subtask block	81
22.10.4	A.9.4 Subtask block	82
22.10.5	A.9.5 Subtask block	82
22.10.6	A.9.6 Subtask block	82
22.10.7	A.9.7 Subtask block	83
22.10.8	A.9.8 Subtask block	83
22.10.9	A.9.9 Subtask block	83
22.10.10	A.9.10 Subtask block	84
22.11	A.10 Safety, watermark, and abuse defense	84
22.11.1	A.10.1 Subtask block	84
22.11.2	A.10.2 Subtask block	84
22.11.3	A.10.3 Subtask block	84
22.11.4	A.10.4 Subtask block	85
22.11.5	A.10.5 Subtask block	85
22.11.6	A.10.6 Subtask block	85
22.11.7	A.10.7 Subtask block	86
22.11.8	A.10.8 Subtask block	86
22.11.9	A.10.9 Subtask block	86
22.11.10	A.10.10 Subtask block	87
22.12	A.11 60-day launch demo	87
22.12.1	A.11.1 Subtask block	87
22.12.2	A.11.2 Subtask block	87
22.12.3	A.11.3 Subtask block	87
22.12.4	A.11.4 Subtask block	87
22.12.5	A.11.5 Subtask block	88

22.12.6A.11.6 Subtask block	88
22.12.7A.11.7 Subtask block	88
22.12.8A.11.8 Subtask block	88
22.12.9A.11.9 Subtask block	88
22.12.10A.11.10 Subtask block	88
22.13A.12 16-week v1 scale-up	89
22.13.1A.12.1 Subtask block	89
22.13.2A.12.2 Subtask block	89
22.13.3A.12.3 Subtask block	89
22.13.4A.12.4 Subtask block	89
22.13.5A.12.5 Subtask block	89
22.13.6A.12.6 Subtask block	90
22.13.7A.12.7 Subtask block	90
22.13.8A.12.8 Subtask block	90
22.13.9A.12.9 Subtask block	90
22.13.10A.12.10 Subtask block	90
22.14A.13 9-month frontier path	90
22.14.1A.13.1 Subtask block	90
22.14.2A.13.2 Subtask block	91
22.14.3A.13.3 Subtask block	91
22.14.4A.13.4 Subtask block	91
22.14.5A.13.5 Subtask block	91
22.14.6A.13.6 Subtask block	91
22.14.7A.13.7 Subtask block	92
22.14.8A.13.8 Subtask block	92
22.14.9A.13.9 Subtask block	92
22.14.10A.13.10 Subtask block	92

23 Appendix B. Agent PR templates	92
23.1 PR template for codec	92
23.2 PR template for tokenizer	93
23.3 PR template for data	93
23.4 PR template for model	93
23.5 PR template for training	94
23.6 PR template for inference	94
23.7 PR template for eval	94
23.8 PR template for safety	94
24 Appendix C. Listening rubric	95
25 Appendix D. Example emotional prompts	95
26 Appendix E. Source-to-decision map	95

1 SOULYATRI VOICE AI - Final Build Bible

Version: v1.0. Prepared on 2026-05-15.

This is intentionally a clean-room plan. It does not tell you to steal private weights, scrape private datasets, impersonate people, reverse engineer protected endpoints, or clone real voices without consent. The goal is to build a better category competitor using public research, open models, consented data, strong engineering, and brutal evaluation.

Line-count note: a literal 1,000,000-line PDF would be unusable and would mostly be padding. This document is built to be agent-executable instead: dense, structured, task-by-task, with every module broken into implementation units, tests, prompts, metrics, and definitions of done.

1.1 One-page decision

Decision	Final call	Why
Product name	Soulyatri Voice AI	Soul + yatra = emotional journey, aligned with companion voice.
Model codename	SVA-1	First shippable model family.
60-day path	Open stack + LoRA/SFT + data engine + streaming wrapper	Fastest credible demo; no from-scratch pretraining.
16-week path	CSM/Moshi-derived TTS + duplex + DPO + code-switch data moat	Real product v1.
9-month path	SVA-Foundation, 3B/8B speech-language model with custom data	Frontier attempt.
Main moat	Code-switched emotional data + evaluation + latency engineering	Architecture alone is not enough.
Safety policy	Consent-only voice cloning + watermark + abuse monitoring	Voice cloning without consent is banned.

\newpage

2 0. Evidence and assumptions

2.1 0.1 What I trust

- Public sources and open repositories listed in the source index.
- Uploaded build plans as useful strategy drafts, not as automatically verified truth.
- The resume screenshot as user-provided context about Vatsal Bharti, not independent verification.
- Metrics only after they are measured by the evaluation harness described here.

2.2 0.2 What I do not trust

- Any unsourced latency, quality, cost, or benchmark claim.
- Any claim that “beat Hume/Silk” is achieved without blind human eval and reproducible logs.
- Any plan that depends on proprietary data or non-consensual voice cloning.
- Any 60-day plan that includes full speech foundation pretraining from scratch.

2.3 0.3 User-provided Vatsal Bharti resume context

- CTO at Rumik.ai, built proprietary voice AI model Silk and scaled product from roughly 60K to 1M MAU in 6 months.
- Expanded technical org from 2 to 12 in 6 months: 8 engineers and 4 AI researchers.
- Built real-time streaming inference for Indian language TTS around 2-3 min ElevenLabs changes and sub-1 second response latency.
- Designed real-time streaming inference with turn detection, low latency handling, backpressure management, and chunking for real Indian environments.
- Google AI Consultant for News and Media India, worked on Gemini rollout into newsrooms and AI workflows for journalists.
- Independent AI research: distillation, quantization, voice model data curation, prosodic annotation, low-resource TTS, long-context retrieval agents.

Implication: a Vatsal-like profile should own architecture, audio tokenization strategy, and real-time latency. A separate data/infra crew should own the dataset flywheel and production serving.

2.4 0.4 Uploaded documents integrated

- VOX_OMEGA_FINAL_VERIFIED_60_DAY_PLAN: strong reality check for 60 days: demo yes, frontier no.
- VOX_OMEGA_MASTER_PLAN: useful 6-month and 9-month structure with CSM/Mimi/EmoVoice/Moshi.
- SILKPP_BUILD_BIBLE: useful agent-executable repo/tree/data/latency breakdown.
- SILKPP_Kill_List_Battle_Plan: useful competitor framing; all claims must be rechecked before marketing.

3 1. Research conclusions

3.1 1.1 Rumik Silk category analysis

Silk 1 is best treated as an expressive companion TTS product: code-switched Hinglish/Tanglish/Manglish/Kanglish, inline vocal events, mid-sentence tone changes, and production latency around hundreds of milliseconds according to the public research page. The core technical pattern is likely a transformer speech generation model over discrete audio codes, not a fundamentally new model class. The category moat is data: messy Romanized Indic text, emotional tags, multi-speaker expressiveness, and evaluation for real companion use.

3.2 1.2 Sesame CSM conclusion

CSM is the clean-room architecture baseline for high-quality conversational speech generation. It uses text and speech tokens, splits generation at the zeroth audio codebook, uses a large multimodal backbone plus a smaller audio decoder, and uses compute amortization by training the depth decoder on a random subset of frames. This makes it the best public starting point for the TTS half of Soulyatri Voice AI.

3.3 1.3 Moshi/Mimi conclusion

Moshi is the clean-room baseline for full-duplex real-time dialogue. Its key idea is to model user audio and model audio as parallel streams, plus an inner monologue text stream. Mimi is the codec that makes this feasible: 24 kHz speech to 12.5 Hz token frames, meaning 80 ms codec frames. For Soulyatri, the first version should use or adapt Mimi; do not retrain a codec in the 60-day path.

3.4 1.4 Hume conclusion

Hume EVI is strongest where most TTS products are weak: prosody understanding, emotional reaction, turn timing, and interruptibility. Beating Hume means not only generating emotional speech, but also understanding the user voice, detecting emotional state, and deciding when to speak. Soulyatri must build an input affect stack, not just an output TTS stack.

3.5 1.5 OpenVoice / voice cloning conclusion

Short-reference voice cloning is technically plausible, but product launch must be consent-only. The correct approach is not arbitrary public cloning. The correct approach is verified user- owned voices, consent phrase, voice embedding ownership checks, watermarking, and rate-limited internal demos until safety passes.

3.6 1.6 The winning thesis

Soulyatri Voice AI wins only if it combines all seven layers: 1. Mimi/codec-token low latency. 2. CSM-style contextual TTS quality. 3. Moshi-style full-duplex turn dynamics. 4. Hume-style input affect understanding. 5. Silk-style Indic code-switch data. 6. EmoVoice-style controllable output emotion. 7. DPO/GRPO/human preference alignment plus rigorous eval.

\newpage

4 2. Product spec

Dimension	Day 30	Day 60	Week 16	Month 9
First audible response	<700 ms	<300 ms	<150 ms	<80 ms in optimized path
End-to-end voice response	<1.2 s	<600 ms	<350 ms	<250 ms
Interruption stop/fade	<500 ms	<250 ms	<180 ms	<120 ms
Code-switch	Hinglish only	Hinglish strong	Hinglish+Tanglish+Telugu-English	10 Indic mixes
Emotion control	5 tags	10 tags + intensity	V/A/D + tags + duration	continuous controllable prosody
Personas	1 rough	3 polished	10 polished	persona marketplace
Voice cloning	off	consent-only same-user	consented 30 sec clone	enterprise verified voice vault
Quality gate	baseline works	A/B beats baseline 60%	A/B beats selected commercial clips in chosen niche	independent benchmark

4.1 2.1 Product modes

4.1.1 Companion mode

Emotionally warm, Hinglish-native, memory-aware, always interruptible.

4.1.2 Creator mode

Generate expressive voiceovers with tags, pauses, laughs, whispers, and persona presets.

4.1.3 Agent mode

Customer support / tutor voice agent with strict safety and low latency.

4.1.4 Studio mode

Offline high-quality rendering; slower but better prosody and acting.

4.1.5 Edge mode

4-bit / 8-codebook fast model for mobile or local demos after v1.

5 3. Architecture

5.1 3.1 Two-track architecture

Track	Purpose	Day-60 implementation	Week-16 implementation
Track A: Duplex	Real-time voice-to-voice	Moshi-style base, minimal fine-tune, interruption wrapper	SVA-Duplex with user/model audio streams and inner monologue
Track B: Modular fallback	Robust shipping path	Streaming VAD/STT + small LLM + CSM/Fish/OpenVoice-like TTS	Hybrid planner with emotion tags and cached TTS context

Never bet the company on one architecture in the first 60 days. Track B ships if Track A breaks.

5.2 3.2 Runtime flow

```

User mic 24 kHz
-> WebRTC ingest
-> streaming VAD + interruption monitor
-> affect extractor: prosody, rhythm, timbre, energy, pitch, speaking rate
-> Track A: speech-to-speech core OR Track B: streaming STT fallback
-> dialog planner produces text + emotion plan + safety constraints
-> emotion-conditioned TTS core
-> codec tokens
-> Mimi-compatible decoder
-> watermark
-> audio chunks over WebRTC

```

5.3 3.3 Model stack

Layer	Day-60 stack	Week-16 stack	Month-9 stack
Codec	frozen Mimi or base model codec	Mimi adapter + RVQ dropout experiments	Mimi-Lite retrain/fine-tune for Indic
Text tokenizer	existing processor + emotion special tokens	PCS-Tok++ prototype	PCS-Tok++ native training
TTS core	CSM-1B/Fish/OpenVoice-like fine-tune	CSM-style 1B/3B with emotion FiLM	SVA-Slow 8B + fast decoder
Duplex	Moshi wrapper	Moshi fine-tune on Hinglish dialogue	SVA-Duplex native
Emotion	tags	tags + intensity + V/A/D embeddings	full FiLM + reward alignment
Serving	FastAPI/WebSocket/WebRTC	LiveKit/Pipewaiter + CUDA graphs	vLLM fork + custom audio sampler
Safety	consent off switch + watermark stub	enforced consent + watermark	full abuse classifier + forensic marks

6 4. Metrics and evaluation

Metric	Definition	Target
TTFA / TTFB	Time from last user audio frame or first text token to first playable assistant audio	<300 ms p50 by Day 60; <150 ms stretch; <80 ms only after kernel work

Metric	Definition	Target
Voice-to-voice end-to-end	User stops speaking to assistant audio start	<600 ms Day 30; <300 ms Day 60; <200 ms stretch
Interruption latency	User interruption to assistant stop/fade	<250 ms Day 60; <180 ms stretch
WER / CER	Back-transcribe audio and compare to intended transcript	WER <8% EN; <12% Hinglish initially; <8% Hinglish v1
Code-switch F1	Correct language/script boundaries and no accent collapse	>0.85 Day 60; >0.92 v1
Emotion controllability	Human label matches requested emotion and intensity	>60% Day 60; >75% v1
CMOS / human preference	Blind A/B against baseline and previous checkpoint	>60% prefer new over baseline; >55% against selected commercial clips
Speaker similarity	ECAPA/WavLM cosine for consented voice demo	>0.72 Day 60; >0.80 v1
Safety block rate	Non-consensual clone attempts blocked	100% known tests; 0 public arbitrary voice clone launch

6.1 4.1 Evaluation harness requirements

- Every generated audio sample must store checkpoint id, prompt, transcript, voice id, emotion request, seed, latency trace, and consent flag.
- Every evaluation run must write a JSONL manifest and a markdown report.
- Every model promotion requires blind A/B listening against the previous checkpoint.
- Every latency claim must include p50, p90, p95, network region, GPU, batch size, and warm/cold cache state.
- Every code-switch claim must include language boundary accuracy and back-transcription errors, not only MOS.

6.2 4.2 Eval JSON schema

```
{
  "sample_id": "sva_eval_000001",
  "checkpoint": "sva_lora_v003",
  "input_text": "arre yaar <laugh> this is actually insane",
  "requested_emotion": {"label": "excited", "intensity": 0.85},
  "voice_id": "consented_voice_001",
  "latency_ms": {"ttfa": 284, "e2e": 512, "interrupt": 210},
  "wer": 0.082,
  "code_switch_f1": 0.89,
  "human_rating": {"naturalness": 4, "emotion_match": 5},
  "safety": {"consent_verified": true, "watermarked": true}
}
```

\newpage

7 5. Repo skeleton

```
soulyatri-voice-ai/
  README.md
  pyproject.toml
  LICENSE
  .python-version
  .pre-commit-config.yaml
  configs/base.yaml
  configs/model/csm_lb_lora.yaml
  configs/model/moshi_duplex.yaml
  configs/model/sva_slow8b.yaml
  configs/model/fast_depth_decoder.yaml
  configs/model/mimi_adapter.yaml
  configs/model/pcs_tok.yaml
```

```

configs/data/tier0_public.yaml
configs/data/tier1_hinglish_text.yaml
configs/data/tier2_emotion_audio.yaml
configs/data/tier3_preference_pairs.yaml
configs/train/stage00_baseline.yaml
configs/train/stage01_tags_lora.yaml
configs/train/stage02_hinglish_sft.yaml
configs/train/stage03_emotion_sft.yaml
configs/train/stage04_dpo_lite.yaml
configs/train/stage05_duplex.yaml
configs/infer/streaming_local.yaml
configs/infer/production_h100.yaml
configs/infer/edge_mlx.yaml
soulyatri/audio/clean.py
soulyatri/audio/vad.py
soulyatri/audio/metrics.py
soulyatri/codec/mimi_adapter.py
soulyatri/codec/rvq_dropout.py
soulyatri/codec/codec_eval.py
soulyatri/tokenizer/pcs_tok.py
soulyatri/tokenizer/phoneme_anchor.py
soulyatri/tokenizer/emotion_grammar.py
soulyatri/data/schema.py
soulyatri/data/synth_hinglish.py
soulyatri/data/filter_audio.py
soulyatri/data/build_lhotse.py
soulyatri/data/build_webdataset.py
soulyatri/model/emotion_film.py
soulyatri/model/speaker_cond.py
soulyatri/model/csm_lora.py
soulyatri/model/duplex_streams.py
soulyatri/training/stage00_baseline.py
soulyatri/training/stage01_lora_tags.py
soulyatri/training/stage02_hinglish_sft.py
soulyatri/training/stage03_emotion_sft.py
soulyatri/training/stage04_dpo.py
soulyatri/training/stage05_grpo.py
soulyatri/inference/server.py
soulyatri/inference/livekit_agent.py
soulyatri/inference/pipecat_pipeline.py
soulyatri/inference/interrupt_monitor.py
soulyatri/inference/latency_tracer.py
soulyatri/eval/wer.py
soulyatri/eval/mos.py
soulyatri/eval/emotion_accuracy.py
soulyatri/eval/code_switch.py
soulyatri/eval/speaker_similarity.py
soulyatri/eval/human_ab.py
soulyatri/safety/consent.py
soulyatri/safety/watermark.py
soulyatri/safety/policy.py
scripts/00_bootstrap.sh
scripts/01_download_weights.sh
scripts/02_prepare_data_dirs.sh
scripts/03_generate_synth_text.sh
scripts/04_synthesize_audio.sh
scripts/05_filter_dataset.sh
scripts/06_train_lora.sh
scripts/07_train_emotion.sh
scripts/08_train_dpo.sh
scripts/09_serve_local.sh
scripts/10_run_eval_suite.sh
tests/test_emotion_grammar.py
tests/test_consent_gate.py
tests/test_latency_budget.py
tests/test_pcs_tokenizer.py
tests/test_streaming_api.py
docs/ARCHITECTURE.md
docs/DATA.md
docs/TRAINING.md
docs/SERVING.md
docs/SAFETY.md

```

8 6. Agent execution protocol

8.1 6.1 Rules for Claude Code / Codex / Cursor

- Never implement a module without adding a test in tests/.
- Never add a dependency without updating pyproject.toml and docs/DEPENDENCIES.md.
- Never change public APIs without updating docs/ARCHITECTURE.md.

- Never merge training code unless it can run a 2-minute smoke test on a tiny sample.
- Never write code that enables public arbitrary voice cloning.
- Never report benchmark wins without a saved eval artifact.

8.2 6.2 Universal agent prompt

You are implementing Soulyatri Voice AI. Read SOULYATRI_VOICE_AI_FINAL_BUILD_BIBLE.md. Implement only the module requested. Create tests. Use type hints. Add docstrings. Do not add non-consensual voice cloning paths. After coding, run the exact smoke test and summarize files changed. If a requirement is impossible, write a TODO with a reason and a minimal fallback.

8.3 6.3 Division of labor

Agent	Owns	Never owns
Claude Code	multi-file refactors, Python modules, tests, docs	CUDA kernels without separate review
GPT Codex	isolated functions, benchmarks, CI, kernel prototypes	product decisions
Cursor	in-repo navigation, UI/server edits	model architecture changes
Human	data approval, eval listening, safety sign-off, architecture	mindless boilerplate

9 7. Module build cards

9.1 7.1 Module: codec - Mimi adapter and future Mimi-Lite

File	Purpose	Unit test	Definition of done
soulyatri/codec/mimi_adapter. ↪ py	Implement mimi adapter and future mimi-lite component mimi_adapter.py	tests/test_mimi_adapter.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/codec/rvq_dropout. ↪ py	Implement mimi adapter and future mimi-lite component rvq_dropout.py	tests/test_rvq_dropout.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/codec/codec_eval.py	Implement mimi adapter and future mimi-lite component codec_eval.py	tests/test_codec_eval.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the codec module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/codec. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.2 7.2 Module: tokenizer - PCS-Tok++ phonetic code-switch tokenizer

File	Purpose	Unit test	Definition of done
soulyatri/tokenizer/pcs_tok. ↪ py	Implement pcs-tok++ phonetic code-switch tokenizer component pcs_tok.py	tests/test_pcs_tok.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/tokenizer/ ↪ phoneme_anchor.py	Implement pcs-tok++ phonetic code-switch tokenizer component phoneme_anchor.py	tests/test_phoneme_anchor.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/tokenizer/ ↪ emotion_grammar.py	Implement pcs-tok++ phonetic code-switch tokenizer component emotion_grammar.py	tests/test_emotion_grammar.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the tokenizer module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/tokenizer. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.3 7.3 Module: data - Synthetic + real data flywheel

File	Purpose	Unit test	Definition of done
soulyatri/data/schema.py	Implement synthetic + real data flywheel component schema.py	tests/test_schema.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/data/synth_hinglish ↪ .py	Implement synthetic + real data flywheel component synth_hinglish.py	tests/test_synth_hinglish.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/data/filter_audio. ↪ py	Implement synthetic + real data flywheel component filter_audio.py	tests/test_filter_audio.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/data/build_lhotse. ↪ py	Implement synthetic + real data flywheel component build_lhotse.py	tests/test_build_lhotse.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/data/ ↪ build_webdataset.py	Implement synthetic + real data flywheel component build_webdataset.py	tests/test_build_webdataset. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the data module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/data. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.4 7.4 Module: model - CSM/Moshi adapters, emotion FiLM, speaker conditioning

File	Purpose	Unit test	Definition of done
soulyatri/model/emotion_film. ↪ py	Implement csm/moshi adapters, emotion film, speaker conditioning component emotion_film.py	tests/test_emotion_film.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/model/speaker_cond. ↪ py	Implement csm/moshi adapters, emotion film, speaker conditioning component speaker_cond.py	tests/test_speaker_cond.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/model/csm_lora.py	Implement csm/moshi adapters, emotion film, speaker conditioning component csm_lora.py	tests/test_csm_lora.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/model/ ↪ duplex_streams.py	Implement csm/moshi adapters, emotion film, speaker conditioning component duplex_streams.py	tests/test_duplex_streams.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the model module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/model. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.5 7.5 Module: training - SFT, DPO, GRPO and smoke tests

File	Purpose	Unit test	Definition of done
soulyatri/training/ ↪ stage00_baseline.py	Implement sft, dpo, grpo and smoke tests component stage00_baseline.py	tests/test_stage00_baseline. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/training/ ↪ stage01_lora_tags.py	Implement sft, dpo, grpo and smoke tests component stage01_lora_tags.py	tests/test_stage01_lora_tags. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors

File	Purpose	Unit test	Definition of done
soulyatri/training/ ↪ stage02_hinglish_sft.py	Implement sft, dpo, grpo and smoke tests component stage02_hinglish_sft.py	tests/ ↪ test_stage02_hinglish_sft. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/training/ ↪ stage03_emotion_sft.py	Implement sft, dpo, grpo and smoke tests component stage03_emotion_sft.py	tests/ ↪ test_stage03_emotion_sft. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/training/ ↪ stage04_dpo.py	Implement sft, dpo, grpo and smoke tests component stage04_dpo.py	tests/test_stage04_dpo.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/training/ ↪ stage05_grpo.py	Implement sft, dpo, grpo and smoke tests component stage05_grpo.py	tests/test_stage05_grpo.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the training module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/training. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.6 7.6 Module: inference - Streaming server and latency engine

File	Purpose	Unit test	Definition of done
soulyatri/inference/server.py	Implement streaming server and latency engine component server.py	tests/test_server.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/inference/ ↪ livekit_agent.py	Implement streaming server and latency engine component livekit_agent.py	tests/test_livekit_agent.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/inference/ ↪ pipecat_pipeline.py	Implement streaming server and latency engine component pipecat_pipeline.py	tests/test_pipecat_pipeline. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/inference/ ↪ interrupt_monitor.py	Implement streaming server and latency engine component interrupt_monitor.py	tests/test_interrupt_monitor. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/inference/ ↪ latency_tracer.py	Implement streaming server and latency engine component latency_tracer.py	tests/test_latency_tracer.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the inference module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/inference. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.7 7.7 Module: eval - Voice quality, code-switch, emotion, safety metrics

File	Purpose	Unit test	Definition of done
soulyatri/eval/wer.py	Implement voice quality, code-switch, emotion, safety metrics component wer.py	tests/test_wer.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/eval/mos.py	Implement voice quality, code-switch, emotion, safety metrics component mos.py	tests/test_mos.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/eval/ ↪ emotion_accuracy.py	Implement voice quality, code-switch, emotion, safety metrics component emotion_accuracy.py	tests/test_emotion_accuracy. ↪ py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/eval/code_switch.py	Implement voice quality, code-switch, emotion, safety metrics component code_switch.py	tests/test_code_switch.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/eval/ ↪ speaker_similarity.py	Implement voice quality, code-switch, emotion, safety metrics component speaker_similarity.py	tests/test_speaker_similarity ↪ .py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/eval/human_ab.py	Implement voice quality, code-switch, emotion, safety metrics component human_ab.py	tests/test_human_ab.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the eval module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/eval. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

9.8 7.8 Module: safety - Consent, watermarking, policy gates

File	Purpose	Unit test	Definition of done
soulyatri/safety/consent.py	Implement consent, watermarking, policy gates component consent.py	tests/test_consent.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/safety/watermark.py	Implement consent, watermarking, policy gates component watermark.py	tests/test_watermark.py	imports cleanly, has typed public API, has smoke test, logs structured errors
soulyatri/safety/policy.py	Implement consent, watermarking, policy gates component policy.py	tests/test_policy.py	imports cleanly, has typed public API, has smoke test, logs structured errors

Claude prompt:

Implement the safety module for Soulyatri Voice AI. Use the file list above. Start with the smallest runnable skeleton, then add tests. Prefer deterministic behavior and JSONL logs. Do not add heavyweight training dependencies inside runtime modules.

Codex prompt:

Write unit tests and benchmarks for soulyatri/safety. Focus on edge cases, malformed inputs, and failure modes. Return a patch only. Do not change public APIs unless tests prove need.

\newpage

10 8. Data engine

10.1 8.1 Dataset tiers

Tier	Purpose	Day-60 target	Week-16 target	Month-9 target
T0	baseline public speech	20-50 hr	5k-20k hr	80k+ hr
T1	Hinglish/code-switch text	100k-500k lines	5M lines	50M lines
T2	synthetic emotional audio	10-30 hr	500-3000 hr	5000+ hr
T3	real emotional studio data	2-5 hr	200-1500 hr	3000+ hr
T4	preference pairs	2k-10k	100k	500k+
T5	safety data	500-2k cases	20k cases	100k cases

10.2 8.2 Synthetic Hinglish prompt template

Generate 50 short voice-assistant utterances in natural Hinglish Latin script.
Audience: Indian companion app users aged 18-35.
Mix: 55 percent Hindi words in Latin script, 45 percent English.
Use one emotion tag per utterance from: <warm>, <excited>, <sad>, <playful>, <sarcastic>, <tired>, <whisper>, <laugh>, <sigh>, <pause ms=200>.
Avoid hate, explicit sexual content, political persuasion, and real-person impersonation.
Return JSONL with fields: text, emotion, intensity, topic, persona, language_mix.

10.3 8.3 Real recording SOP

- Prepare 500 prompts across warm, excited, sad, playful, frustrated, whisper, laugh, sigh.
- Collect explicit written consent with usage scope: training, evaluation, demo, revocation path.
- Record 24 kHz or 48 kHz WAV, mono, fixed mic distance, no clipping, peak below -3 dBFS.
- Capture actor metadata: age band, accent region, gender expression if voluntarily provided, language comfort.
- Record 30 seconds of neutral consent phrase for voice verification.
- Run automatic filters: VAD, SNR, clipping, ASR back-transcription, duration, duplicate detection.
- Human spot-check 10 percent of samples; reject an actor batch if more than 10 percent fail.
- Store manifest with consent id and deletion handle for revocation.

10.4 8.4 Data manifest schema

```
sample_id: sva_real_000001
audio_path: s3://soulyatri-data/clean/sva_real_000001.wav
text: "arre yaar, this actually made my day"
language_mix: [hi-Latn, en]
emotion: warm
emotion_intensity: 0.72
speaker_id: actor_023
consent_id: consent_023_2026_05_15
source: real_studio
sample_rate: 24000
duration_sec: 3.2
quality:
  snr_db: 31.0
  wer: 0.04
  clipping: false
  human_accepted: true
```

11 9. Training plan

11.1 9.1 Day-60 training sequence

Stage	Days	Action	GPU	Gate
0	1-3	Run base model inference and latency baseline	1 GPU	talks end-to-end
1	4-10	Build data pipeline and real micro-set	CPU/API	manifest passes QA
2	11-18	Tag-only LoRA SFT	1 A100/H100 or 2x4090	emotion tags audible
3	19-26	Scale synthetic data and retrain	1 GPU	WER/MOS stable
4	27-35	Streaming wrapper and interruption	1 GPU	<400 ms p50 demo
5	36-44	DPO-lite preferences	1 GPU	A/B beats SFT
6	45-52	Personas and consent flow	1 GPU	3 personas stable
7	53-60	Latency polish and launch	1 GPU	public/private demo

11.2 9.2 Hyperparameter starting defaults

```
lora_rank: 32
lora_alpha: 64
```

```
sft_lr: 2.0e-5
dpo_lr: 3.0e-7
batching: gradient_accumulation_until_gpu_full
precision: bf16
freeze: codec_decoder, most_backbone_blocks
unfreeze: emotion_tokens, adapters, output_projection_if_needed
loss: token_nll_plus_optional_emotion_aux_after_baseline
max_audio_seconds_initial: 12
max_audio_seconds_v2: 30
```

11.3 9.3 DPO pair schema

```
{
  "prompt": "<warm intensity=0.7> kal ka interview kaisa gaya?",
  "chosen_audio": "s3://pairs/chosen_001.wav",
  "rejected_audio": "s3://pairs/rejected_001.wav",
  "rubric": {
    "naturalness": "chosen",
    "emotion_match": "chosen",
    "intelligibility": "tie",
    "code_switch": "chosen"
  },
  "raters": 5,
  "agreement": 0.8
}
```

12 10. Latency engineering

12.1 10.1 Budget

Component	Day-60 target	Week-16 target	Tactics
Mic/WebRTC/VAD	40 ms	25 ms	small chunks, minimal jitter buffer
Turn detection	60 ms	35 ms	prosody-aware endpointing, no long silence wait
Dialog/model planning	120 ms	60 ms	warm KV, small model, prompt prefill
TTS first audio	90 ms	50 ms	chunked decode, CUDA graphs, static cache
Network/playback	50 ms	30 ms	same-region hosting, WebRTC tuning
Total	<300 ms	<200 ms	measure p50/p95 on real calls

12.2 10.2 Optimization order

1. Remove architecture hops: speech-to-speech or streaming TTS beats batch TTS.
2. Warm everything: model weights, tokenizer, persona prompt, first KV cache, CUDA graphs.
3. Emit audio before sentence completion; use phoneme/chunk lookahead.
4. Keep context short in the critical path; push memory retrieval to async.
5. Use quantization and torch.compile/CUDA graphs after correctness.
6. Only then write custom Triton/CUDA fused audio-codebook kernels.

12.3 10.3 Latency tracer event list

- mic_frame_received

- vad_speech_start
- vad_speech_end
- asr_partial
- dialog_prefill_start
- dialog_first_token
- tts_prefill_start
- audio_token_first
- audio_pcm_first
- webrtc_send_first
- client_play_first
- interrupt_detected
- assistant_stop_signal
- assistant_audio_zero

\newpage

13 11. Safety and consent

13.1 11.1 Non-negotiable rules

- No public arbitrary voice cloning.
- No cloning a real person unless they have explicitly consented and passed verification.
- No political persuasion or deceptive robocalls.
- No impersonation, fraud, fake emergency calls, or synthetic evidence generation.
- Generated audio must carry a watermark or persistent metadata marker where feasible.
- Every voice has a revocation path and deletion handle.

13.2 11.2 Consent gate pseudocode

```
def can_generate_with_voice(user_id: str, voice_id: str, request: dict) -> bool:
    voice = voice_registry.get(voice_id)
    if voice is None:
        return False
    if voice.owner_user_id != user_id and not voice.enterprise_delegate_allowed:
        return False
    if not voice.consent_active:
        return False
    if request.get("public_clone") is True:
        return False
    if policy_classifier.is_deceptive_impersonation(request):
        return False
    return True
```

14 12. Phase-by-phase execution

14.1 Phase 0: Reality-lock and target definition

Duration: 1 day

Goal: Make the target measurable, not motivational.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report

Workstream	Tasks	Owner	DoD
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 0 objective: Make the target measurable, not motivational.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 0.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 0.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 0.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 0.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 0.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 0.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 0.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 0.8: complete subtask 8, attach logs, update eval if relevant.

14.2 Phase 1: Repo, environment, and baseline demo

Duration: 3 days

Goal: Get a base voice model speaking end-to-end before any training.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 1 objective: Get a base voice model speaking end-to-end before any training.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 1.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 1.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 1.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 1.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 1.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 1.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 1.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 1.8: complete subtask 8, attach logs, update eval if relevant.

14.3 Phase 2: Evaluation harness

Duration: 4 days

Goal: Every future claim must be measured automatically.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 2 objective: Every future claim must be measured automatically.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 2.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 2.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 2.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 2.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 2.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 2.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 2.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 2.8: complete subtask 8, attach logs, update eval if relevant.

14.4 Phase 3: Emotion grammar and text data engine

Duration: 7 days

Goal: Generate clean Hinglish/emotion text with metadata and safety filters.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 3 objective: Generate clean Hinglish/emotion text with metadata and safety filters.. Break this into a pull request
 ↳ with tests,
 docs, and an eval artifact. Do not touch unrelated files. If blocked, create
 a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 3.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 3.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 3.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 3.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 3.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 3.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 3.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 3.8: complete subtask 8, attach logs, update eval if relevant.

14.5 Phase 4: Consent-first real recording micro-set

Duration: 7 days

Goal: Record 2-5 hours of real expressive audio with signed consent.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report

Workstream	Tasks	Owner	DoD
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 4 objective: Record 2-5 hours of real expressive audio with signed consent.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 4.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 4.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 4.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 4.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 4.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 4.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 4.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 4.8: complete subtask 8, attach logs, update eval if relevant.

14.6 Phase 5: Tag-only LoRA SFT

Duration: 8 days

Goal: Make emotion tags audible and stable with minimal VRAM.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 5 objective: Make emotion tags audible and stable with minimal VRAM.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 5.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 5.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 5.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 5.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 5.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 5.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 5.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 5.8: complete subtask 8, attach logs, update eval if relevant.

14.7 Phase 6: Hinglish/code-switch SFT v2

Duration: 8 days

Goal: Make Romanized Indian code-switching natural.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 6 objective: Make Romanized Indian code-switching natural.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 6.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 6.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 6.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 6.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 6.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 6.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 6.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 6.8: complete subtask 8, attach logs, update eval if relevant.

14.8 Phase 7: Streaming product shell

Duration: 10 days

Goal: WebRTC in/out, live interruption, latency tracing.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 7 objective: WebRTC in/out, live interruption, latency tracing.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 7.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 7.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 7.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 7.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 7.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 7.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 7.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 7.8: complete subtask 8, attach logs, update eval if relevant.

14.9 Phase 8: DPO-lite alignment

Duration: 10 days

Goal: A/B preference tuning for naturalness and emotional appropriateness.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report

Workstream	Tasks	Owner	DoD
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 8 objective: A/B preference tuning for naturalness and emotional appropriateness.. Break this into a pull request
 ↳ with tests,
 docs, and an eval artifact. Do not touch unrelated files. If blocked, create
 a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 8.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 8.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 8.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 8.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 8.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 8.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 8.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 8.8: complete subtask 8, attach logs, update eval if relevant.

14.10 Phase 9: Persona and voice conditioning

Duration: 8 days

Goal: Build 3-5 consented personas that sound alive.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 9 objective: Build 3-5 consented personas that sound alive.. Break this into a pull request with tests,
 docs, and an eval artifact. Do not touch unrelated files. If blocked, create
 a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 9.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 9.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 9.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 9.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 9.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 9.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 9.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 9.8: complete subtask 8, attach logs, update eval if relevant.

14.11 Phase 10: Safety, watermark, and abuse defense

Duration: 5 days

Goal: Block non-consensual cloning and mark generated audio.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 10 objective: Block non-consensual cloning and mark generated audio.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 10.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 10.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 10.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 10.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 10.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 10.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 10.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 10.8: complete subtask 8, attach logs, update eval if relevant.

14.12 Phase 11: 60-day launch demo

Duration: 9 days

Goal: Private beta demo with metrics, clips, landing page, and investor story.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 11 objective: Private beta demo with metrics, clips, landing page, and investor story.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 11.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 11.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 11.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 11.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 11.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 11.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 11.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 11.8: complete subtask 8, attach logs, update eval if relevant.

14.13 Phase 12: 16-week v1 scale-up

Duration: 8 weeks

Goal: Scale data, teams, serving, and product integrations.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report

Workstream	Tasks	Owner	DoD
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 12 objective: Scale data, teams, serving, and product integrations.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 12.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 12.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 12.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 12.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 12.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 12.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 12.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 12.8: complete subtask 8, attach logs, update eval if relevant.

14.14 Phase 13: 9-month frontier path

Duration: 5 months

Goal: Train or expand foundation-quality stack, not just fine-tune.

Workstream	Tasks	Owner	DoD
Architecture	architecture plan, implementation, smoke test, review, artifact log	Founder/ML lead	Architecture artifact merged, tested, and referenced in eval report
Data	data plan, implementation, smoke test, review, artifact log	Data engineer	Data artifact merged, tested, and referenced in eval report
Model	model plan, implementation, smoke test, review, artifact log	ML engineer	Model artifact merged, tested, and referenced in eval report
Serving	serving plan, implementation, smoke test, review, artifact log	Infra/FE	Serving artifact merged, tested, and referenced in eval report
Eval	eval plan, implementation, smoke test, review, artifact log	Research/eval	Eval artifact merged, tested, and referenced in eval report
Safety	safety plan, implementation, smoke test, review, artifact log	Founder/legal	Safety artifact merged, tested, and referenced in eval report

Agent prompt:

Phase 13 objective: Train or expand foundation-quality stack, not just fine-tune.. Break this into a pull request with tests, docs, and an eval artifact. Do not touch unrelated files. If blocked, create a minimal fallback and explain the tradeoff in docs/DECISIONS.md.

Checklist: - [] Phase 13.1: complete subtask 1, attach logs, update eval if relevant. - [] Phase 13.2: complete subtask 2, attach logs, update eval if relevant. - [] Phase 13.3: complete subtask 3, attach logs, update eval if relevant. - [] Phase 13.4: complete subtask 4, attach logs, update eval if relevant. - [] Phase 13.5: complete subtask 5, attach logs, update eval if relevant. - [] Phase 13.6: complete subtask 6, attach logs, update eval if relevant. - [] Phase 13.7: complete subtask 7, attach logs, update eval if relevant. - [] Phase 13.8: complete subtask 8, attach logs, update eval if relevant.

\newpage

15 13. Day-by-day 60-day calendar

15.1 Day 1 - Week 1

- ☐ D1.1 Create repo and issue tracker.
- ☐ D1.2 Rent GPU.
- ☐ D1.3 Download base weights.
- ☐ D1.4 Run base demo.
- ☐ D1.5 Record baseline latency.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.2 Day 2 - Week 1

- ☐ D2.1 Baseline evaluation.
- ☐ D2.2 Data schema.
- ☐ D2.3 Synthetic text generator.
- ☐ D2.4 Real micro-recording.
- ☐ D2.5 Audio cleaning.
- ☐ D2.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.3 Day 3 - Week 1

- ☐ D3.1 Baseline evaluation.
- ☐ D3.2 Data schema.
- ☐ D3.3 Synthetic text generator.
- ☐ D3.4 Real micro-recording.
- ☐ D3.5 Audio cleaning.
- ☐ D3.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.4 Day 4 - Week 1

- ☐ D4.1 Baseline evaluation.
- ☐ D4.2 Data schema.
- ☐ D4.3 Synthetic text generator.
- ☐ D4.4 Real micro-recording.
- ☐ D4.5 Audio cleaning.
- ☐ D4.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.5 Day 5 - Week 1

- ☐ D5.1 Baseline evaluation.
- ☐ D5.2 Data schema.
- ☐ D5.3 Synthetic text generator.
- ☐ D5.4 Real micro-recording.
- ☐ D5.5 Audio cleaning.
- ☐ D5.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.6 Day 6 - Week 1

- ☐ D6.1 Baseline evaluation.
- ☐ D6.2 Data schema.
- ☐ D6.3 Synthetic text generator.
- ☐ D6.4 Real micro-recording.
- ☐ D6.5 Audio cleaning.
- ☐ D6.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.7 Day 7 - Week 1

- ☐ D7.1 Baseline evaluation.
- ☐ D7.2 Data schema.
- ☐ D7.3 Synthetic text generator.
- ☐ D7.4 Real micro-recording.
- ☐ D7.5 Audio cleaning.
- ☐ D7.6 Back-transcription filter.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.8 Day 8 - Week 2

- ☐ D8.1 Emotion vocabulary.
- ☐ D8.2 Tokenizer handling.
- ☐ D8.3 LoRA training script.
- ☐ D8.4 30-minute smoke fine-tune.
- ☐ D8.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.9 Day 9 - Week 2

- ☐ D9.1 Emotion vocabulary.
- ☐ D9.2 Tokenizer handling.
- ☐ D9.3 LoRA training script.
- ☐ D9.4 30-minute smoke fine-tune.
- ☐ D9.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.10 Day 10 - Week 2

- ☐ D10.1 Emotion vocabulary.

- ☐ D10.2 Tokenizer handling.
- ☐ D10.3 LoRA training script.
- ☐ D10.4 30-minute smoke fine-tune.
- ☐ D10.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.11 Day 11 - Week 2

- ☐ D11.1 Emotion vocabulary.
- ☐ D11.2 Tokenizer handling.
- ☐ D11.3 LoRA training script.
- ☐ D11.4 30-minute smoke fine-tune.
- ☐ D11.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.12 Day 12 - Week 2

- ☐ D12.1 Emotion vocabulary.
- ☐ D12.2 Tokenizer handling.
- ☐ D12.3 LoRA training script.
- ☐ D12.4 30-minute smoke fine-tune.
- ☐ D12.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.13 Day 13 - Week 2

- ☐ D13.1 Emotion vocabulary.
- ☐ D13.2 Tokenizer handling.
- ☐ D13.3 LoRA training script.
- ☐ D13.4 30-minute smoke fine-tune.
- ☐ D13.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.14 Day 14 - Week 2

- ☐ D14.1 Emotion vocabulary.
- ☐ D14.2 Tokenizer handling.
- ☐ D14.3 LoRA training script.
- ☐ D14.4 30-minute smoke fine-tune.
- ☐ D14.5 Debug tags and pauses.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.15 Day 15 - Week 3

- ☐ D15.1 Scale synthetic data.
- ☐ D15.2 Train v2.
- ☐ D15.3 Add personas.
- ☐ D15.4 Build web demo shell.
- ☐ D15.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.16 Day 16 - Week 3

- ☐ D16.1 Scale synthetic data.
- ☐ D16.2 Train v2.
- ☐ D16.3 Add personas.
- ☐ D16.4 Build web demo shell.
- ☐ D16.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.17 Day 17 - Week 3

- ☐ D17.1 Scale synthetic data.
- ☐ D17.2 Train v2.
- ☐ D17.3 Add personas.
- ☐ D17.4 Build web demo shell.
- ☐ D17.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.18 Day 18 - Week 3

- ☐ D18.1 Scale synthetic data.
- ☐ D18.2 Train v2.
- ☐ D18.3 Add personas.
- ☐ D18.4 Build web demo shell.
- ☐ D18.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.19 Day 19 - Week 3

- ☐ D19.1 Scale synthetic data.
- ☐ D19.2 Train v2.
- ☐ D19.3 Add personas.
- ☐ D19.4 Build web demo shell.
- ☐ D19.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.20 Day 20 - Week 3

- ☐ D20.1 Scale synthetic data.
- ☐ D20.2 Train v2.
- ☐ D20.3 Add personas.
- ☐ D20.4 Build web demo shell.
- ☐ D20.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.21 Day 21 - Week 3

- ☐ D21.1 Scale synthetic data.
- ☐ D21.2 Train v2.
- ☐ D21.3 Add personas.
- ☐ D21.4 Build web demo shell.

- ☐ D21.5 Stream chunks.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.22 Day 22 - Week 4

- ☐ D22.1 Interruption monitor.
- ☐ D22.2 Stop/fade tuning.
- ☐ D22.3 Persona balancing.
- ☐ D22.4 Latency pass.
- ☐ D22.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.23 Day 23 - Week 4

- ☐ D23.1 Interruption monitor.
- ☐ D23.2 Stop/fade tuning.
- ☐ D23.3 Persona balancing.
- ☐ D23.4 Latency pass.
- ☐ D23.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.24 Day 24 - Week 4

- ☐ D24.1 Interruption monitor.
- ☐ D24.2 Stop/fade tuning.
- ☐ D24.3 Persona balancing.
- ☐ D24.4 Latency pass.
- ☐ D24.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.25 Day 25 - Week 4

- ☐ D25.1 Interruption monitor.
- ☐ D25.2 Stop/fade tuning.
- ☐ D25.3 Persona balancing.
- ☐ D25.4 Latency pass.
- ☐ D25.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.26 Day 26 - Week 4

- ☐ D26.1 Interruption monitor.
- ☐ D26.2 Stop/fade tuning.
- ☐ D26.3 Persona balancing.
- ☐ D26.4 Latency pass.
- ☐ D26.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.27 Day 27 - Week 4

- ☐ D27.1 Interruption monitor.
- ☐ D27.2 Stop/fade tuning.
- ☐ D27.3 Persona balancing.
- ☐ D27.4 Latency pass.
- ☐ D27.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.28 Day 28 - Week 4

- ☐ D28.1 Interruption monitor.
- ☐ D28.2 Stop/fade tuning.
- ☐ D28.3 Persona balancing.
- ☐ D28.4 Latency pass.
- ☐ D28.5 Private demo and ratings.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.29 Day 29 - Week 5

- ☐ D29.1 Generate preference pairs.
- ☐ D29.2 Human A/B labeling.
- ☐ D29.3 DPO dataset build.
- ☐ D29.4 DPO smoke run.
- ☐ D29.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.30 Day 30 - Week 5

- ☐ D30.1 Generate preference pairs.
- ☐ D30.2 Human A/B labeling.
- ☐ D30.3 DPO dataset build.
- ☐ D30.4 DPO smoke run.
- ☐ D30.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.31 Day 31 - Week 5

- ☐ D31.1 Generate preference pairs.
- ☐ D31.2 Human A/B labeling.
- ☐ D31.3 DPO dataset build.
- ☐ D31.4 DPO smoke run.
- ☐ D31.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.32 Day 32 - Week 5

- ☐ D32.1 Generate preference pairs.
- ☐ D32.2 Human A/B labeling.
- ☐ D32.3 DPO dataset build.
- ☐ D32.4 DPO smoke run.

- ☐ D32.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.33 Day 33 - Week 5

- ☐ D33.1 Generate preference pairs.
- ☐ D33.2 Human A/B labeling.
- ☐ D33.3 DPO dataset build.
- ☐ D33.4 DPO smoke run.
- ☐ D33.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.34 Day 34 - Week 5

- ☐ D34.1 Generate preference pairs.
- ☐ D34.2 Human A/B labeling.
- ☐ D34.3 DPO dataset build.
- ☐ D34.4 DPO smoke run.
- ☐ D34.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.35 Day 35 - Week 5

- ☐ D35.1 Generate preference pairs.
- ☐ D35.2 Human A/B labeling.
- ☐ D35.3 DPO dataset build.
- ☐ D35.4 DPO smoke run.
- ☐ D35.5 Failure analysis.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.36 Day 36 - Week 6

- ☐ D36.1 Train DPO-lite.
- ☐ D36.2 Evaluate emotion match.
- ☐ D36.3 Improve weak emotions.
- ☐ D36.4 Retune data mix.
- ☐ D36.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.37 Day 37 - Week 6

- ☐ D37.1 Train DPO-lite.
- ☐ D37.2 Evaluate emotion match.
- ☐ D37.3 Improve weak emotions.
- ☐ D37.4 Retune data mix.
- ☐ D37.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.38 Day 38 - Week 6

- ☐ D38.1 Train DPO-lite.
- ☐ D38.2 Evaluate emotion match.
- ☐ D38.3 Improve weak emotions.
- ☐ D38.4 Retune data mix.
- ☐ D38.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.39 Day 39 - Week 6

- ☐ D39.1 Train DPO-lite.
- ☐ D39.2 Evaluate emotion match.
- ☐ D39.3 Improve weak emotions.
- ☐ D39.4 Retune data mix.
- ☐ D39.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.40 Day 40 - Week 6

- ☐ D40.1 Train DPO-lite.
- ☐ D40.2 Evaluate emotion match.
- ☐ D40.3 Improve weak emotions.
- ☐ D40.4 Retune data mix.
- ☐ D40.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.41 Day 41 - Week 6

- ☐ D41.1 Train DPO-lite.
- ☐ D41.2 Evaluate emotion match.
- ☐ D41.3 Improve weak emotions.
- ☐ D41.4 Retune data mix.
- ☐ D41.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.42 Day 42 - Week 6

- ☐ D42.1 Train DPO-lite.
- ☐ D42.2 Evaluate emotion match.
- ☐ D42.3 Improve weak emotions.
- ☐ D42.4 Retune data mix.
- ☐ D42.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.43 Day 43 - Week 7

- ☐ D43.1 Train DPO-lite.
- ☐ D43.2 Evaluate emotion match.
- ☐ D43.3 Improve weak emotions.
- ☐ D43.4 Retune data mix.

- ☐ D43.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.44 Day 44 - Week 7

- ☐ D44.1 Train DPO-lite.
- ☐ D44.2 Evaluate emotion match.
- ☐ D44.3 Improve weak emotions.
- ☐ D44.4 Retune data mix.
- ☐ D44.5 Checkpoint promotion.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.45 Day 45 - Week 7

- ☐ D45.1 Consent flow.
- ☐ D45.2 Voice ownership checks.
- ☐ D45.3 Watermark integration.
- ☐ D45.4 Three personas polished.
- ☐ D45.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.46 Day 46 - Week 7

- ☐ D46.1 Consent flow.
- ☐ D46.2 Voice ownership checks.
- ☐ D46.3 Watermark integration.
- ☐ D46.4 Three personas polished.
- ☐ D46.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.47 Day 47 - Week 7

- ☐ D47.1 Consent flow.
- ☐ D47.2 Voice ownership checks.
- ☐ D47.3 Watermark integration.
- ☐ D47.4 Three personas polished.
- ☐ D47.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.48 Day 48 - Week 7

- ☐ D48.1 Consent flow.
- ☐ D48.2 Voice ownership checks.
- ☐ D48.3 Watermark integration.
- ☐ D48.4 Three personas polished.
- ☐ D48.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.49 Day 49 - Week 7

- ☐ D49.1 Consent flow.
- ☐ D49.2 Voice ownership checks.
- ☐ D49.3 Watermark integration.
- ☐ D49.4 Three personas polished.
- ☐ D49.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.50 Day 50 - Week 8

- ☐ D50.1 Consent flow.
- ☐ D50.2 Voice ownership checks.
- ☐ D50.3 Watermark integration.
- ☐ D50.4 Three personas polished.
- ☐ D50.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.51 Day 51 - Week 8

- ☐ D51.1 Consent flow.
- ☐ D51.2 Voice ownership checks.
- ☐ D51.3 Watermark integration.
- ☐ D51.4 Three personas polished.
- ☐ D51.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.52 Day 52 - Week 8

- ☐ D52.1 Consent flow.
- ☐ D52.2 Voice ownership checks.
- ☐ D52.3 Watermark integration.
- ☐ D52.4 Three personas polished.
- ☐ D52.5 Safety tests.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.53 Day 53 - Week 8

- ☐ D53.1 Latency polish.
- ☐ D53.2 p95 debugging.
- ☐ D53.3 Landing page.
- ☐ D53.4 Final eval.
- ☐ D53.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.54 Day 54 - Week 8

- ☐ D54.1 Latency polish.
- ☐ D54.2 p95 debugging.
- ☐ D54.3 Landing page.
- ☐ D54.4 Final eval.

- ☐ D54.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.55 Day 55 - Week 8

- ☐ D55.1 Latency polish.
- ☐ D55.2 p95 debugging.
- ☐ D55.3 Landing page.
- ☐ D55.4 Final eval.
- ☐ D55.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.56 Day 56 - Week 8

- ☐ D56.1 Latency polish.
- ☐ D56.2 p95 debugging.
- ☐ D56.3 Landing page.
- ☐ D56.4 Final eval.
- ☐ D56.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.57 Day 57 - Week 9

- ☐ D57.1 Latency polish.
- ☐ D57.2 p95 debugging.
- ☐ D57.3 Landing page.
- ☐ D57.4 Final eval.
- ☐ D57.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.58 Day 58 - Week 9

- ☐ D58.1 Latency polish.
- ☐ D58.2 p95 debugging.
- ☐ D58.3 Landing page.
- ☐ D58.4 Final eval.
- ☐ D58.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.59 Day 59 - Week 9

- ☐ D59.1 Latency polish.
- ☐ D59.2 p95 debugging.
- ☐ D59.3 Landing page.
- ☐ D59.4 Final eval.
- ☐ D59.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

15.60 Day 60 - Week 9

- ☐ D60.1 Latency polish.
- ☐ D60.2 p95 debugging.
- ☐ D60.3 Landing page.
- ☐ D60.4 Final eval.
- ☐ D60.5 Private beta launch prep.
- ☐ Save artifacts: logs, generated audio, eval JSONL, and decision note.
- ☐ End-of-day rule: no unmeasured claim goes into pitch or docs.

\newpage

16 14. File-by-file implementation backlog

16.1 Backlog item 1: soulyatri/codec/mimi_adapter.py

Field	Value
Purpose	Mimi adapter and future Mimi-Lite
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.2 Backlog item 2: soulyatri/codec/rvq_dropout.py

Field	Value
Purpose	Mimi adapter and future Mimi-Lite
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.3 Backlog item 3: soulyatri/codec/codec_eval.py

Field	Value
Purpose	Mimi adapter and future Mimi-Lite
Inputs	typed dataclasses or JSONL records

Field	Value
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.4 Backlog item 4: soulyatri/tokenizer/pcs_tok.py

Field	Value
Purpose	PCS-Tok++ phonetic code-switch tokenizer
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.5 Backlog item 5: soulyatri/tokenizer/phoneme_anchor.py

Field	Value
Purpose	PCS-Tok++ phonetic code-switch tokenizer
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.6 Backlog item 6: soulyatri/tokenizer/emotion_grammar.py

Field	Value
Purpose	PCS-Tok++ phonetic code-switch tokenizer
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs

Field	Value
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.7 Backlog item 7: soulyatri/data/schema.py

Field	Value
Purpose	Synthetic + real data flywheel
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.8 Backlog item 8: soulyatri/data/synth_hinglish.py

Field	Value
Purpose	Synthetic + real data flywheel
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.9 Backlog item 9: soulyatri/data/filter_audio.py

Field	Value
Purpose	Synthetic + real data flywheel
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration

Field	Value
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.10 Backlog item 10: soulyatri/data/build_lhotse.py

Field	Value
Purpose	Synthetic + real data flywheel
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.11 Backlog item 11: soulyatri/data/build_webdataset.py

Field	Value
Purpose	Synthetic + real data flywheel
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.12 Backlog item 12: soulyatri/model/emotion_film.py

Field	Value
Purpose	CSM/Moshi adapters, emotion FiLM, speaker conditioning
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data

Field	Value
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.13 Backlog item 13: soulyatri/model/speaker_cond.py

Field	Value
Purpose	CSM/Moshi adapters, emotion FiLM, speaker conditioning
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.14 Backlog item 14: soulyatri/model/csm_lora.py

Field	Value
Purpose	CSM/Moshi adapters, emotion FiLM, speaker conditioning
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.15 Backlog item 15: soulyatri/model/duplex_streams.py

Field	Value
Purpose	CSM/Moshi adapters, emotion FiLM, speaker conditioning
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path

Field	Value
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.16 Backlog item 16: soulyatri/training/stage00_baseline.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.17 Backlog item 17: soulyatri/training/stage01_lora_tags.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.18 Backlog item 18: soulyatri/training/stage02_hinglish_sft.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.19 Backlog item 19: soulyatri/training/stage03_emotion_sft.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.20 Backlog item 20: soulyatri/training/stage04_dpo.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.21 Backlog item 21: soulyatri/training/stage05_grpo.py

Field	Value
Purpose	SFT, DPO, GRPO and smoke tests
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.22 Backlog item 22: soulyatri/inference/server.py

Field	Value
Purpose	Streaming server and latency engine
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.23 Backlog item 23: soulyatri/inference/livekit_agent.py

Field	Value
Purpose	Streaming server and latency engine
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.24 Backlog item 24: soulyatri/inference/pipecat_pipeline.py

Field	Value
Purpose	Streaming server and latency engine
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.25 Backlog item 25: soulyatri/inference/interrupt_monitor.py

Field	Value
Purpose	Streaming server and latency engine
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.26 Backlog item 26: soulyatri/inference/latency_tracer.py

Field	Value
Purpose	Streaming server and latency engine
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.27 Backlog item 27: soulyatri/eval/wer.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.28 Backlog item 28: soulyatri/eval/mos.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics

Field	Value
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.29 Backlog item 29: soulyatri/eval/emotion_accuracy.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.30 Backlog item 30: soulyatri/eval/code_switch.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.31 Backlog item 31: soulyatri/eval/speaker_similarity.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics
Inputs	typed dataclasses or JSONL records

Field	Value
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.32 Backlog item 32: soulyatri/eval/human_ab.py

Field	Value
Purpose	Voice quality, code-switch, emotion, safety metrics
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.33 Backlog item 33: soulyatri/safety/consent.py

Field	Value
Purpose	Consent, watermarking, policy gates
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.34 Backlog item 34: soulyatri/safety/watermark.py

Field	Value
Purpose	Consent, watermarking, policy gates
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs

Field	Value
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

16.35 Backlog item 35: soulyatri/safety/policy.py

Field	Value
Purpose	Consent, watermarking, policy gates
Inputs	typed dataclasses or JSONL records
Outputs	typed result object plus structured logs
Tests	unit, malformed input, smoke integration
Failure handling	raise typed error; never silently skip data
Performance	add benchmark if runtime path
Security	no external network calls unless explicitly configured

Subtasks: - [] create typed config - [] write core function - [] add validation - [] add JSONL logging - [] write unit tests - [] write smoke test - [] document examples - [] add TODOs for future scale

\newpage

17 15. Copy-paste commands

17.1 15.1 Bootstrap

```
git init soulyatri-voice-ai
cd soulyatri-voice-ai
python3.11 -m venv .venv
source .venv/bin/activate
pip install -U pip uv
uv pip install torch torchaudio transformers accelerate datasets
uv pip install lhotse webdataset fastapi uvicorn websockets pydantic
uv pip install pytest ruff mypy rich typer wandb
mkdir -p soulyatri/{audio,codec,tokenizer,data,model,training,inference,eval,safety}
mkdir -p configs/{model,data,train,infer} scripts tests docs
```

17.2 15.2 Baseline eval

```
python -m soulyatri.training.stage00_baseline \
  --model sesame/csm-lb \
  --eval_manifest data/eval/tiny.jsonl \
  --out runs/baseline_001
python -m soulyatri.eval.leaderboard --run runs/baseline_001
```

17.3 15.3 LoRA SFT

```
accelerate launch soulyatri/training/stage01_lora_tags.py \
  --config configs/train/stage01_tags_lora.yaml \
  --train data/manifests/emotion_train.jsonl \
  --valid data/manifests/emotion_valid.jsonl \
  --out checkpoints/sval_lora_tags_v001
```

17.4 15.4 Serve local demo

```
python -m soulyatri.inference.server \
  --checkpoint checkpoints/sval_lora_tags_v001 \
  --config configs/infer/streaming_local.yaml \
  --port 7860
```

18 16. Budget

Path	Time	Team	Compute/data budget	Outcome
Solo demo	60 days	1	\$2k-\$15k	impressive private demo, not frontier
Serious MVP	16 weeks	3-5	\$75k-\$250k	strong niche product
Funded v1	6 months	7-10	\$400k-\$900k	credible Hume/Silk competitor in chosen niche
Frontier foundation	9-12 months	12-20	\$1.5M-\$5M+	independent benchmark attempt

19 17. Risk register

Risk	Likelihood	Impact	Mitigation
Data quality is weak	High	High	Small real studio set, human QA, aggressive filtering
Latency misses target	High	High	Track B fallback, warm cache, smaller model, region co-location
Emotion sounds acted/fake	High	Medium	DPO pairs and real emotional micro-data
Hinglish pronunciation breaks	High	High	PCS tokenizer, back-transcription, human accent eval
Voice cloning misuse	High	High	Consent gate, watermark, no public arbitrary clone
Model licensing blocks commercial use	Medium	High	License audit before dependency; commercial fallback plan
GPU budget explodes	Medium	High	LoRA path first, no scratch pretraining until funded
Eval is subjective	High	Medium	Blind A/B, rater agreement, saved samples

20 18. Source index

- [R1] Rumik Silk 1 research page: <https://rumik.ai/research/silk> - Silk 1 positioning, code-switching problem, data scarcity, inline emotion tags, transformer/audio-token architecture, latency claim.
- [R2] Sesame CSM research: https://www.sesame.com/research/crossing_the_uncanny_valley_of_voice - Two-transformer CSM architecture, split at codebook zero, Mimi RVQ tokens, compute amortization, evaluation gaps.
- [R3] SesameAILabs/csm GitHub: <https://github.com/SesameAILabs/csm> - Open CSM-1B implementation, Llama backbone, smaller audio decoder, Mimi codes, fine-tuning path, misuse constraints.
- [R4] Kyutai Moshi GitHub: <https://github.com/kyutai-labs/moshi> - Full-duplex speech-text model, Mimi codec, two audio streams, inner monologue, practical 200 ms latency claim.
- [R5] Kyutai Mimi model card: <https://huggingface.co/kyutai/mimi> - Mimi codec: 12.5 Hz, 1.1 kbps, streaming encoder-decoder, speech codec use.
- [R6] Moshi paper: <https://arxiv.org/abs/2410.00037> - Speech-to-speech framework, parallel user/model streams, latency and tokenized speech formulation.
- [R7] EmoVoice paper: <https://arxiv.org/abs/2504.12867> - Freestyle natural-language emotional TTS prompting, phoneme and audio token design, 40-hour emotional dataset.
- [R8] OpenVoice paper: <https://arxiv.org/abs/2312.01479> - Short-reference voice cloning and flexible style controls; use only with explicit consent.
- [R9] Hume EVI 3 launch post: <https://www.hume.ai/blog/introducing-evi-3> - Speech-language model, any prompt-created voice/personality, RL voice-quality tuning, latency and eval claims.
- [R10] Hume EVI docs: <https://dev.hume.ai/docs/speech-to-speech-evi/overview> - EVI measures vocal modulations, prosody, turn timing, interruptibility, WebSocket architecture, supported languages.
- [R11] Fish Speech GitHub: <https://github.com/fishaudio/fish-speech> - S2 Pro claims: Dual-AR, RL alignment, large multilingual audio training base.
- [R12] vLLM docs: <https://docs.vllm.ai/en/latest/> - PagedAttention, continuous batching, prefix caching, CUDA graphs, quantization, speculative decoding, streaming outputs.
- [R13] PagedAttention paper: <https://arxiv.org/abs/2309.06180> - KV-cache memory management for high-throughput LLM serving.
- [R14] SpeechAlign paper: <https://arxiv.org/abs/2404.05600> - Preference alignment for speech generation.
- [R15] Not My Voice taxonomy: <https://arxiv.org/abs/2402.01708> - Ethical and safety harms of speech generators; consent and misuse guardrails.

21 19. Final operating principle

Do not build the biggest model first. Build the most emotionally believable 3-minute live demo first. Then use the demo to earn data, users, capital, and the right to train the bigger model.

\newpage

22 Appendix A. Ultra-granular phase subtasks

22.1 A.0 Reality-lock and target definition

22.1.1 A.0.1 Subtask block

- ☐ A.0.1.1: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.1.2: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.1.3: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.1.4: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.1.5: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.

- ☐ A.0.8.3: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.8.4: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.8.5: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.8.6: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.8.7: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.

22.1.9 A.0.9 Subtask block

- ☐ A.0.9.1: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.2: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.3: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.4: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.5: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.6: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.9.7: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.

22.1.10 A.0.10 Subtask block

- ☐ A.0.10.1: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.2: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.3: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.4: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.5: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.6: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.
- ☐ A.0.10.7: define input, implement, test, log, review, and document one atomic unit for Reality-lock and target definition.

22.2 A.1 Repo, environment, and baseline demo

22.2.1 A.1.1 Subtask block

- ☐ A.1.1.1: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.1.2: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.1.3: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.1.4: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.

- ☐ A.1.8.3: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.8.4: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.8.5: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.8.6: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.8.7: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.

22.2.9 A.1.9 Subtask block

- ☐ A.1.9.1: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.2: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.3: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.4: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.5: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.6: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.9.7: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.

22.2.10 A.1.10 Subtask block

- ☐ A.1.10.1: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.2: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.3: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.4: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.5: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.6: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.
- ☐ A.1.10.7: define input, implement, test, log, review, and document one atomic unit for Repo, environment, and baseline demo.

22.3 A.2 Evaluation harness

22.3.1 A.2.1 Subtask block

- ☐ A.2.1.1: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.2: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.3: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.4: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.5: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.6: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.1.7: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.

- ☐ A.2.7.6: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.7.7: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.

22.3.8 A.2.8 Subtask block

- ☐ A.2.8.1: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.2: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.3: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.4: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.5: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.6: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.8.7: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.

22.3.9 A.2.9 Subtask block

- ☐ A.2.9.1: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.2: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.3: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.4: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.5: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.6: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.9.7: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.

22.3.10 A.2.10 Subtask block

- ☐ A.2.10.1: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.2: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.3: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.4: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.5: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.6: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.
- ☐ A.2.10.7: define input, implement, test, log, review, and document one atomic unit for Evaluation harness.

22.4 A.3 Emotion grammar and text data engine

22.4.1 A.3.1 Subtask block

- ☐ A.3.1.1: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.2: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.3: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.4: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.5: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.6: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.1.7: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.

22.4.2 A.3.2 Subtask block

- A.3.2.1: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.

22.4.9 A.3.9 Subtask block

- ☐ A.3.9.1: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.2: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.3: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.4: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.5: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.6: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.9.7: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.

22.4.10 A.3.10 Subtask block

- ☐ A.3.10.1: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.2: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.3: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.4: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.5: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.6: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.
- ☐ A.3.10.7: define input, implement, test, log, review, and document one atomic unit for Emotion grammar and text data engine.

22.5 A.4 Consent-first real recording micro-set

22.5.1 A.4.1 Subtask block

- ☐ A.4.1.1: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.2: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.3: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.4: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.5: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.6: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.1.7: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.

22.5.2 A.4.2 Subtask block

- ☐ A.4.2.1: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.

22.5.9 A.4.9 Subtask block

- ☐ A.4.9.1: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.2: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.3: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.4: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.5: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.6: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.9.7: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.

22.5.10 A.4.10 Subtask block

- ☐ A.4.10.1: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.2: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.3: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.4: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.5: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.6: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.
- ☐ A.4.10.7: define input, implement, test, log, review, and document one atomic unit for Consent-first real recording micro-set.

22.6 A.5 Tag-only LoRA SFT

22.6.1 A.5.1 Subtask block

- ☐ A.5.1.1: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.2: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.3: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.4: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.5: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.6: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.1.7: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.

22.6.2 A.5.2 Subtask block

- ☐ A.5.2.1: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.2: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.3: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.4: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.5: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.6: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.2.7: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.

- ☐ A.5.8.6: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.8.7: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.

22.6.9 A.5.9 Subtask block

- ☐ A.5.9.1: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.2: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.3: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.4: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.5: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.6: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.9.7: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.

22.6.10 A.5.10 Subtask block

- ☐ A.5.10.1: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.2: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.3: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.4: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.5: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.6: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.
- ☐ A.5.10.7: define input, implement, test, log, review, and document one atomic unit for Tag-only LoRA SFT.

22.7 A.6 Hinglish/code-switch SFT v2

22.7.1 A.6.1 Subtask block

- ☐ A.6.1.1: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.2: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.3: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.4: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.5: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.6: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.1.7: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.

22.7.2 A.6.2 Subtask block

- ☐ A.6.2.1: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.2.2: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.2.3: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.2.4: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.2.5: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.2.6: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.

- ☐ A.6.9.3: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.9.4: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.9.5: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.9.6: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.9.7: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.

22.7.10 A.6.10 Subtask block

- ☐ A.6.10.1: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.2: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.3: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.4: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.5: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.6: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.
- ☐ A.6.10.7: define input, implement, test, log, review, and document one atomic unit for Hinglish/code-switch SFT v2.

22.8 A.7 Streaming product shell

22.8.1 A.7.1 Subtask block

- ☐ A.7.1.1: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.2: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.3: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.4: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.5: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.6: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.1.7: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.

22.8.2 A.7.2 Subtask block

- ☐ A.7.2.1: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.2.2: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.2.3: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.
- ☐ A.7.2.4: define input, implement, test, log, review, and document one atomic unit for Streaming product shell.

- ☐ A.8.9.3: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.9.4: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.9.5: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.9.6: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.9.7: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.

22.9.10 A.8.10 Subtask block

- ☐ A.8.10.1: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.2: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.3: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.4: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.5: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.6: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.
- ☐ A.8.10.7: define input, implement, test, log, review, and document one atomic unit for DPO-lite alignment.

22.10 A.9 Persona and voice conditioning

22.10.1 A.9.1 Subtask block

- ☐ A.9.1.1: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.2: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.3: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.4: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.5: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.6: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.1.7: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.

22.10.2 A.9.2 Subtask block

- ☐ A.9.2.1: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.2: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.3: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.4: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.5: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.6: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.2.7: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.

22.10.3 A.9.3 Subtask block

- ☐ A.9.3.1: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.

22.10.10 A.9.10 Subtask block

- ☐ A.9.10.1: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.2: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.3: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.4: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.5: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.6: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.
- ☐ A.9.10.7: define input, implement, test, log, review, and document one atomic unit for Persona and voice conditioning.

22.11 A.10 Safety, watermark, and abuse defense

22.11.1 A.10.1 Subtask block

- ☐ A.10.1.1: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.2: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.3: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.4: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.5: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.6: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.1.7: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.

22.11.2 A.10.2 Subtask block

- ☐ A.10.2.1: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.2: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.3: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.4: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.5: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.6: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.
- ☐ A.10.2.7: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.

22.11.3 A.10.3 Subtask block

- ☐ A.10.3.1: define input, implement, test, log, review, and document one atomic unit for Safety, watermark, and abuse defense.

22.14.7 A.13.7 Subtask block

- ☐ A.13.7.1: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.2: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.3: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.4: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.5: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.6: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.7.7: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.

22.14.8 A.13.8 Subtask block

- ☐ A.13.8.1: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.2: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.3: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.4: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.5: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.6: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.8.7: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.

22.14.9 A.13.9 Subtask block

- ☐ A.13.9.1: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.2: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.3: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.4: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.5: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.6: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.9.7: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.

22.14.10 A.13.10 Subtask block

- ☐ A.13.10.1: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.2: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.3: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.4: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.5: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.6: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.
- ☐ A.13.10.7: define input, implement, test, log, review, and document one atomic unit for 9-month frontier path.

\newpage

23 Appendix B. Agent PR templates

23.1 PR template for codec

```
# PR: codec - <short title>
## What changed
-
## Why
```

```
- Supports Mimi adapter and future Mimi-Lite.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.2 PR template for tokenizer

```
# PR: tokenizer - <short title>

## What changed
-

## Why
- Supports PCS-Tok++ phonetic code-switch tokenizer.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.3 PR template for data

```
# PR: data - <short title>

## What changed
-

## Why
- Supports Synthetic + real data flywheel.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.4 PR template for model

```
# PR: model - <short title>

## What changed
-

## Why
- Supports CSM/Moshi adapters, emotion FiLM, speaker conditioning.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.5 PR template for training

```
# PR: training - <short title>

## What changed
-

## Why
- Supports SFT, DPO, GRPO and smoke tests.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.6 PR template for inference

```
# PR: inference - <short title>

## What changed
-

## Why
- Supports Streaming server and latency engine.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.7 PR template for eval

```
# PR: eval - <short title>

## What changed
-

## Why
- Supports Voice quality, code-switch, emotion, safety metrics.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
- [ ] no non-consensual voice cloning path
- [ ] no unwatermarked public generation path
- [ ] no private data committed
```

23.8 PR template for safety

```
# PR: safety - <short title>

## What changed
-

## Why
- Supports Consent, watermarking, policy gates.

## Tests
- [ ] pytest tests/
- [ ] smoke command pasted below
- [ ] eval artifact path

## Safety
```

- [] no non-consensual voice cloning path
- [] no unwatermarked public generation path
- [] no private data committed

24 Appendix C. Listening rubric

Criterion	Question	Scale
Naturalness	Does it sound human or robotic?	1 robotic, 3 acceptable, 5 human-like
Emotion match	Does it match requested emotion?	1 wrong, 3 partial, 5 exact
Intensity control	Does requested intensity map to perceived intensity?	1 ignored, 3 rough, 5 precise
Code-switch fluency	Does Hinglish flow naturally?	1 broken, 3 understandable, 5 native
Accent retention	Does Indian accent stay natural in English words?	1 collapsed, 3 mixed, 5 natural
Turn timing	Does it start/stop at the right time?	1 awkward, 3 okay, 5 effortless
Companion vibe	Would you keep talking to it?	1 no, 3 maybe, 5 yes

25 Appendix D. Example emotional prompts

- <warm intensity=0.7> arre yaar, tu tension mat le. main hoon na.
- <excited intensity=0.9> bro this is actually insane, humne kar diya!
- <sad intensity=0.6><pause ms=300> mujhe pata hai, aaj ka din heavy tha.
- <playful intensity=0.8><laugh soft dur=0.4> tu phir se late? classic you.
- <whisper intensity=0.5> sun, ek secret bolun? this could actually work.
- <sarcastic intensity=0.65> haan haan, because production bugs obviously weekend pe hi aate hain.

26 Appendix E. Source-to-decision map

Decision	Sources
Use CSM-style TTS core	R2, R3
Use Mimi-compatible codec	R4, R5, R6
Use Moshi-style full-duplex track	R4, R6
Add natural-language emotion control	R7, R9, R10
Add consent-only voice cloning	R8, R15
Use vLLM concepts for serving	R12, R13
Treat data as the moat	R1 plus uploaded build plans